KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Narayanaguda, Hyderabad–500029.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB MANUAL

NAME OF THE LAB: Software Testing Methodologies

B.Tech IV YEAR I SEM (KR23)
ACADEMIC YEAR 2026-27

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

Certificate

This is to certify that following is a Bonafide Record of the workbook task done by

_____bearing Roll No_____of

_____Branch of_____year B.Tech Course in the

_____Subject during the Academic year_____&_____ under our supervision.

Number of experiments completed:_____

Signature of Staff Member Incharge

Signature of Head of the Dept.

Signature of Internal Examiner

Signature of External Examiner

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

Daily Laboratory Assessment Sheet

Name of the Lab: Name of the Student:

Class: HT.No:

S.No.	Name of the Experiment	Date	Observation Marks (3M)	Record Marks (4M)	Viva Voice Marks (3M)	Total Marks (10M)	Signature of Faculty
	TOTAL						

CONTENT

SYLLABUS

1. List of Experiments:

1. Recording in context sensitive mode and analog mode
2. GUI checkpoint for single property
3. GUI checkpoint for single object/window
4. GUI checkpoint for multiple objects
    a) Bitmap checkpoint for object/window
    b) Bitmap checkpoint for screen area
5. Database checkpoint for Default check
6. Database checkpoint for custom check
7. Database checkpoint for runtime record check
    a) Data driven test for dynamic test data submission
    b) Data driven test through flat files
    c) Data driven test through front grids
    d) Data driven test through excel test
    e) Batch testing without parameter passing
    f) Batch testing with parameter passing
8. Data driven batch
9. Silent mode test execution without any interruption
10. Test case for calculator in windows application

TEXT BOOKS:

l. software Testing techniques - Baris Beizer, Dreamtech, second edition.

2. Software Testing Tools — Dr. K. V. K. K. Prasad, Dreamtech.

REFERENCE BOOKS:

I , The craft of software testing - Brian Marick, Pearson Education.

2. Software Testing Techniques — SPD(Oreille)

3. Software Testing in the Real World — Edward Kit, Pearson.

4. Effective methods of Software Testing, Perry, John Wiley.

5. Art of Software Testing — Meyers, John Wiley.

Software to be used: The students must use Selenium.

1. Install Java JDK 11/17 (or higher) and set JAVA_HOME and Path.
2. Install Eclipse IDE for Java Developers.
3. Create a Maven Project in Eclipse (File > New > Maven Project).
4. In pom.xml, add Selenium dependency (selenium-java 4.x) and save.
5. Right-click project > Maven > Update Project (Force Update).
6. Create package: src/main/java > com.test. selenium.
7. Ensure Google Chrome or Microsoft Edge is installed and updated.

Note: Selenium Manager (Selenium 4.6+) usually downloads the correct driver automatically. So you normally do not need to download chromedriver.exe or set System.setProperty for drivers.

Course Objectives: The course will help to

1. To provide knowledge of Software Testing Methods.
2. To develop skills in software test automation and management using latest tools.
3. To expose the advanced software testing topics, such as object-oriented software testing methods.
4. To develop skills in software test automation and management using latest tools.
5. To discuss various software testing issues and solutions in software unit test, integration and system testing.

Course Outcomes: After learning the concepts of this course, the student is able to

1. Design and develop the best test strategies in accordance to the development model.
2. Outline the necessity of testing, debugging using program control flow.
3. Apply transaction flow, data flow testing to unit and integration testing. Able to test a domain or an application and identify the nice and ugly domains
4. Understand the testing of state graphs.
5. Analyze graph matrices for optimizing the code and use of testing tools like WinRunner, JMeter.

Department of Computer Science and Engineering

Vision of the Institution:

To be the fountain head of latest technologies, producing highly skilled, globally competent engineers.

Mission of the Institution:

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepare students to become successful professionals.
- To establish Industry Institute Interaction to make students ready for the industry.
- To provide exposure to students on latest hardware and software tools.
- To promote research based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students

Department of Computer Science and Engineering

Vision of the Department:

To be among the region's premier teaching and research Computer Science and Engineering departments producing globally competent and socially responsible graduates in the most conducive academic environment.

Mission of the Department:

- To provide faculty with state-of-the-art facilities for continuous professional development and research, both in foundational aspects and of relevance to emerging computing trends.
- To impart skills that transform students to develop technical solutions for societal needs and inculcate entrepreneurial talents.
- To inculcate an ability in students to pursue the advancement of knowledge in various specializations of Computer Science and Engineering and make them industry-ready.
- To engage in collaborative research with academia and industry and generate adequate resources for research activities for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To cultivate responsibility through sharing of knowledge and innovative computing solutions that benefits the society-at-large.
- To collaborate with academia, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities.

Department of Computer Science and Engineering

PROGRAM OUTCOMES (POs):

PO1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineeringproblems.

PO2: Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics,natural sciences, and engineering sciences.

PO3: Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and Team Work: Function effectively as an individual, and as a member orleader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend

and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Computer Science solutions for social up liftments.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep Learning, Internet of Things (IOT), Data Science, Full stack development, Social Networks, Cyber Security, Big Data, Mobile Apps, CRM, ERP etc.

Department of Computer Science and Engineering

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will have successful careers in computer related engineering fields or will beable to successfully pursue advanced higher education degrees.

PEO2: Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

PEO3: Graduates will engage in life-long learning and professional development by rapidly adapting changing work environment.

PEO4: Graduates will communicate effectively, work collaboratively and exhibit high levels of professionalism and ethical responsibility.

PROGRAMS

Week1.

Experiment 1: Recording in Context Sensitive Mode and Analog Mode (Selenium IDE)

Tools/Requirements:

- Selenium IDE extension (Chrome/Edge)
- Any demo site for drag-and-drop (optional)

Procedure:

1. Open Chrome/Edge and install the Selenium IDE extension from the browser extensions store.
2. Click the Extensions icon and open Selenium IDE.
3. Click 'Create a new project' and enter a project name.
4. Click 'Add new test' and name the test case (e.g., TC_Record_Google).
5. Click the red 'Record' button.
6. Enter the base URL (e.g., https://www.google.com) and click 'Start Recording'.
7. Perform actions: click the search box, type a keyword, press Enter, click a result, then go back.
8. Stop recording and review the generated commands (open, click, type, assert).
9. Run the test using the 'Run current test' (play) button.
10. For analog-style actions: try recording a drag-and-drop or slider movement on a demo site and replay it.
11. Save the project in Selenium IDE.

Notes:

- Selenium IDE is mainly context-sensitive (element based). True analog recording is limited.
- For complex gestures use Selenium WebDriver code with the Actions class.

Output:

Running 'TC_Record_Google'
open | https://www.google.com |
type | name=q | selenium
assertTitle | Google |
Test completed successfully

Result: PASS

Week2

Experiment 2: GUI Checkpoint for Single Property (Title/Text/Attribute)

Tools/Requirements:
• Selenium WebDriver (Java)
• TestNG or JUnit (recommended)
Procedure:
1. Create a new Java class in package com.test.selenium (e.g., Exp2_SinglePropertyCheckpoint).
2. Launch the browser using Selenium WebDriver and open the target URL.
3. Choose one GUI property to verify (page title OR element text OR an attribute).
4. Capture the actual value (driver.getTitle() / element.getText() / element.getAttribute()).
5. Compare with expected value using assertion (TestNG/JUnit) or if-else logic.
6. Print PASS/FAIL in the console.
7. Close the browser.
Notes:
• Use WebDriverWait if the page/property loads slowly.

Sample Java Code:

```
// Exp 2: Single Property Checkpoint (Page Title)
package com.test.selenium;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class Exp2_SinglePropertyCheckpoint {
   public static void main(String[] args) throws InterruptedException {
      WebDriver driver = new ChromeDriver();
      driver.get("https://www.google.com");

      String expectedTitle = "Google";
      String actualTitle = driver.getTitle();

      System.out.println("Actual Title   : " + actualTitle);
      System.out.println("Expected Title : " + expectedTitle);

      if (actualTitle.equals(expectedTitle)) {
         System.out.println("PASS: Title is correct");
      } else {
         System.out.println("FAIL: Title is incorrect");
      }

      Thread.sleep(2000);
      driver.quit();
   }
}
```

Output

```
Actual Title   : Google
Expected Title : Google
PASS: Title is correct
```

Week3

Experiment 3: GUI Checkpoint for Single Object/Window (Displayed/Enabled)

Tools/Requirements:
• Selenium WebDriver (Java)
Procedure:
1. Create a new Java class (e.g., Exp3_SingleObjectCheckpoint).
2. Launch browser and open the URL.
3. Identify the target element and choose a stable locator (id/name/css/xpath).
4. Use WebDriverWait to wait until the element is present/visible.
5. Verify one condition (isDisplayed / isEnabled / isSelected).
6. Print PASS/FAIL and close the browser.
Notes:
• Avoid Thread.sleep for synchronization; prefer explicit waits.

Sample Java Code:

```
// Exp 3: Single Object Checkpoint (Element displayed and enabled)
package com.test.selenium;

import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;

public class Exp3_SingleObjectCheckpoint {
   public static void main(String[] args) throws InterruptedException {
      WebDriver driver = new ChromeDriver();
      driver.get("https://www.google.com");

      WebElement searchBox = driver.findElement(By.name("q"));
      System.out.println("Displayed: " + searchBox.isDisplayed());
      System.out.println("Enabled  : " + searchBox.isEnabled());

      System.out.println(searchBox.isDisplayed() && searchBox.isEnabled()
         ? "PASS: Search box is ready"
         : "FAIL: Search box is not ready");

      Thread.sleep(2000);
      driver.quit();
   }

}
```

Week4

Experiment 4(a): Bitmap Checkpoint for Object/Window (Screenshot Comparison)

Algorithm / Steps

1. Start the program.
2. Launch the Chrome browser.
3. Open the Google homepage.
4. Locate the Google Search Box using its name attribute (q).
5. Create a folder named **Screenshots** if it does not already exist.
6. Capture the screenshot of the search box object.
7. Save the screenshot as current_searchbox.png.
8. Check whether baseline_searchbox.png exists.
9. If the baseline image is not found:
    - Display a message asking the user to rename current_searchbox.png to baseline_searchbox.png.
10. If the baseline image exists:
    - Compare the baseline and current images pixel by pixel.
11. If both images are identical:
    - Display **BITMAP CHECKPOINT : TEST PASS**.
12. Otherwise:
    - Display **BITMAP CHECKPOINT : TEST FAIL**.
13. Close the browser.

```
package com.test.selenium;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.io.FileHandler;
import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.io.File;
public class NewTest{
   public static void main(String[] args) throws Exception {
      WebDriver driver = new ChromeDriver();
      try {
         driver.get("https://www.google.com");
         // Locate object
         WebElement searchBox = driver.findElement(By.name("q"));
         // Create Screenshots folder
         File folder = new File("Screenshots");
         if (!folder.exists()) {
            folder.mkdir();
         }
         // Capture current screenshot
         File currentImage =
               searchBox.getScreenshotAs(OutputType.FILE);
         File currentFile =
               new File("Screenshots/current_searchbox.png");
         FileHandler.copy(currentImage, currentFile);
```

```
            System.out.println("Current screenshot saved:");
            System.out.println(currentFile.getAbsolutePath());
            // Baseline image
            File baselineFile =
                  new File("Screenshots/baseline_searchbox.png");
            if (!baselineFile.exists()) {
               System.out.println("Baseline image not found.");
               System.out.println("Rename current_searchbox.png as baseline_searchbox.png");
            }
            else {
               boolean result = compareImages(baselineFile, currentFile);
             if(result) {
                System.out.println("BITMAP CHECKPOINT : TEST PASS");
                 System.out.println("Images are identical.")
  }
               else {
              System.out.println("BITMAP CHECKPOINT : TEST FAIL");
             System.out.println("Images are different.");
                  }
            }

         }
catch(Exception e) {
            System.out.println("Execution Failed");
            e.printStackTrace();
         }

         finally {
        driver.quit();
        }    }

    // Method to compare image pixels
   public static boolean compareImages(File img1, File img2)
            throws Exception {
         BufferedImage image1 =   ImageIO.read(img1);
         BufferedImage image2 =   ImageIO.read(img2);
     if(image1.getWidth()!=image2.getWidth()|| image1.getHeight()!=image2.getHeight())
{
 return false;
         }
  for(int x=0; x<image1.getWidth(); x++)
{
for(int y=0; y<image1.getHeight(); y++)
{
          if(image1.getRGB(x,y) != image2.getRGB(x,y)
  return false;
                }
}
```

```
        }
   return true;

    }

}
```

First Excecution

Current screenshot saved:
C:\Users\YourName\eclipse-workspace\YourProject\Screenshots\current_searchbox.png
Baseline image not found.
Rename current_searchbox.png as baseline_searchbox.png

Second Execution

Current screenshot saved:
C:\Users\YourName\eclipse-workspace\YourProject\Screenshots\current_searchbox.png
BITMAP CHECKPOINT : TEST PASS
Images are identical.

Week5

Experiment 4(b): Bitmap Checkpoint for Screen Area (Crop + Compare)
Algorithm / Steps

1. Start the program.
2. Launch the Chrome browser.
3. Maximize the browser window.
4. Open the Google homepage.
5. Wait for 3 seconds to allow the page to load completely.
6. Capture a screenshot of the entire browser window.
7. Convert the screenshot into a BufferedImage.
8. Create a folder named C:\SeleniumScreenshots if it does not already exist.
9. Crop the top-left screen area of size **800 × 600 pixels**.
10. Save the cropped image as current_google_area.png.
11. Check whether baseline_google_area.png exists.
12. If the baseline image does not exist:
    - Display a message asking the user to rename current_google_area.png as baseline_google_area.png.
13. If the baseline image exists:
    - Compare the baseline image and the current image pixel by pixel.
    - Calculate the similarity percentage.
14. If the similarity is **95% or more**, display **TEST PASS**.
15. Otherwise, display **TEST FAIL**.
16. Close the browser.
17. End the program.

```
import javax.imageio.ImageIO;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
public class NewTest {
public static void main(String[] args) throws Exception {
WebDriver driver = new ChromeDriver();
try {
 driver.manage().window().maximize();
driver.get("https://www.google.com");
Thread.sleep(3000);
 // Capture screenshot
        File screenshot =((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
BufferedImage fullImage =  ImageIO.read(screenshot);
// Local folder
  File folder = new File("C:\\SeleniumScreenshots");
  if (!folder.exists())
 {
    folder.mkdirs();
```

```
          }
 // Capture screen area
   int width = Math.min(800, fullImage.getWidth());
   int height = Math.min(600, fullImage.getHeight());
  BufferedImage currentArea =  fullImage.getSubimage(0,0,width,height);
    File currentFile = new File( "C:\\SeleniumScreenshots\\current_google_area.png");
 ImageIO.write(currentArea, "png",    currentFile);
 System.out.println( "Current screenshot saved:");
 System.out.println(currentFile.getAbsolutePath());
File baselineFile =  new File("C:\\SeleniumScreenshots\\baseline_google_area.png");
   if(!baselineFile.exists()) {
System.out.println("Baseline image not found.");
 System.out.println("Rename current_google_area.png to baseline_google_area.png");
   }
else {
  BufferedImage baseline = ImageIO.read(baselineFile);
double similarity =compareImages(baseline, currentArea);
System.out.println(  "Image Similarity : "  + similarity + "%");
 if(similarity >= 95) {
System.out.println(     "BITMAP CHECKPOINT : TEST PASS");
    }
else {
System.out.println("BITMAP CHECKPOINT : TEST FAIL");
 }
 }

       }
       finally {
  driver.quit();
   }
  }
public static double compareImages( BufferedImage img1,BufferedImage img2)
{
 if(img1.getWidth()!=img2.getWidth() || img1.getHeight()!=img2.getHeight()) {
return 0;
       }
 long totalPixels = img1.getWidth()* img1.getHeight();
long samePixels = 0;
for(int x=0; x<img1.getWidth(); x++)
 {
 for(int y=0; y<img1.getHeight(); y++)
 {
   if(img1.getRGB(x,y) == img2.getRGB(x,y))
 {
```

```
samePixels++;

            }

          }
       }
 return
       ((double)samePixels / totalPixels) * 100;

   }

}
```

Output

First Execution

Current screenshot saved:
C:\SeleniumScreenshots\current_google_area.png
Baseline image not found.

Rename current_google_area.png to baseline_google_area.png

Second Execution
Current screenshot saved:
C:\SeleniumScreenshots\current_google_area.png

Image Similarity : 99.72%

BITMAP CHECKPOINT : TEST PASS

Week6

Experiment 5: Database Checkpoint – Default Check (Record Exists)

```
****************************************
// Not For Students (for your refrence)
To install H2 DATABASE Dependency for pom.xml
<dependency>
   <groupId>com.h2database</groupId>
   <artifactId>h2</artifactId>
   <version>2.4.240</version>
</dependency>
****************************************

package com.test.selenium;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class NewTest {
public static void main(String[] args) {
try {
  // Connect to H2 Database
        Connection con = DriverManager.getConnection("jdbc:h2:./StudentDB","sa","");
                Statement stmt = con.createStatement();
// Create table if it does not exist
        stmt.execute("CREATE TABLE IF NOT EXISTS STUDENT ("+ "ID INT PRIMARY
KEY, "+ "NAME VARCHAR(50),"+ "EMAIL VARCHAR(100))");
  // Remove previous data (optional)
        stmt.execute("DELETE FROM STUDENT");
 // Insert one record
        stmt.executeUpdate("INSERT INTO STUDENT VALUES " +
"(1,'Ajay','ajay@gmail.com')");
 // Database Checkpoint
        ResultSet rs = stmt.executeQuery("SELECT * FROM STUDENT WHERE ID = 1");
 if (rs.next()) {
            System.out.println("Record Exists");
            System.out.println("ID    : " + rs.getInt("ID"));
            System.out.println("Name  : " + rs.getString("NAME"));
            System.out.println("Email : " + rs.getString("EMAIL"));
            System.out.println("TEST PASSED");
        } else {
            System.out.println("Record Not Found");
            System.out.println("TEST FAILED");
        }
```

```
// Close connection
            con.close();
} catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output

```
Record Exists
ID    : 1
Name  : Ajay
Email : ajay@gmail.com
TEST PASSED

Process finished with exit code 0
```

Week7

Experiment 6: Database Checkpoint – Custom Check (Validate Specific Value)

```
package com.test.selenium;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class NewTest{
 public static void main(String[] args) {

        try {

            // Connect to H2 Database
            Connection con = DriverManager.getConnection(
                    "jdbc:h2:./StudentDB", "sa", "");

            Statement stmt = con.createStatement();

            // Create table if it does not exist
            stmt.execute("CREATE TABLE IF NOT EXISTS STUDENT ("
                    + "ID INT PRIMARY KEY, "
                    + "NAME VARCHAR(50), "
                    + "EMAIL VARCHAR(100))");

            // Delete previous records
            stmt.execute("DELETE FROM STUDENT");

            // Insert sample record
            stmt.executeUpdate(
                    "INSERT INTO STUDENT VALUES "
                    + "(1,'Ajay','ajay@gmail.com')");

            // Retrieve the record
            ResultSet rs = stmt.executeQuery(
                    "SELECT NAME FROM STUDENT WHERE ID = 1");

            if (rs.next()) {

                String actualName = rs.getString("NAME");
                String expectedName = "Ajay";

                // Custom Check
                if (actualName.equals(expectedName)) {

                   System.out.println("Expected Name : " + expectedName);
                    System.out.println("Actual Name   : " + actualName);
                    System.out.println("CUSTOM CHECK PASSED");
```

```
            } else {

                System.out.println("Expected Name : " + expectedName);
                System.out.println("Actual Name   : " + actualName);
                System.out.println("CUSTOM CHECK FAILED");
            }

        } else {

            System.out.println("Record Not Found");
        }

        con.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

Output

```
Expected Name : Ajay
Actual Name   : Ajay
CUSTOM CHECK PASSED

Process finished with exit code 0
```

Week8

Experiment 7(a): Data Driven Test – Dynamic Test Data Submission

Create Html file

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Registration</title>
</head>
<body>
<h2>Student Registration Form</h2>
<label>Name:</label>
<input type="text" id="name"><br><br>
<label>Email:</label>
<input type="text" id="email"><br><br>
<button onclick="submitForm()">Submit</button>
<p id="result"></p>
<script>
function submitForm(){
var name=document.getElementById("name").value;
var email=document.getElementById("email").value;
document.getElementById("result").innerHTML=
"Registration Successful";
}
</script>
</body>
</html>
```

Save the Html file in the location which is in BOLD in program

```
package com.test.selenium;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
public class DynamicDataSubmission {
  public static void main(String[] args)
        throws Exception {

      WebDriver driver = new ChromeDriver();
```

```
        // Open local HTML page
        driver.get("file:///C:/SeleniumFiles/StudentForm.html");

        // Dynamic Test Data
        String[][] data = {


{"Ajay","ajay@gmail.com"},
                {"Rahul","rahul@gmail.com"},
                {"Anita","anita@gmail.com"}

        };

        for(int i=0;i<data.length;i++)
 {

            WebElement name =  driver.findElement(By.id("name"));

            WebElement email =  driver.findElement(By.id("email"));
             name.clear();
                      email.clear();
            name.sendKeys(data[i][0]);
            email.sendKeys(data[i][1]);

            driver.findElement(By.tagName("button")).click();

            String result = driver.findElement(
                  By.id("result")).getText();

            System.out.println("-----------------------");
            System.out.println("Test Case : "+(i+1));
            System.out.println("Name : "+data[i][0]);
            System.out.println("Email : "+data[i][1]);
            System.out.println(result);
  if(result.equals("Registration Successful"))
               System.out.println("TEST PASS");
            else
               System.out.println("TEST FAIL");
```

```
   Thread.sleep(1000);
    }
      driver.quit();

   }
}
```

Output

```
Test Case : 1
Name : Ajay
Email : ajay@gmail.com
Registration Successful
TEST PASS
----------------------


Test Case : 2
Name : Rahul
Email : rahul@gmail.com
Registration Successful
TEST PASS
----------------------
Test Case : 3
Name : Anita
Email : anita@gmail.com
Registration Successful
TEST PASS
```

Experiment 7(b): Data Driven Test – Through Flat Files (CSV/TXT)

Step 1: Create a HTML File(StudentForm.html)

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Registration Form</title>
</head>
<body>
<h2>Student Registration Form</h2>
<label>Name :</label>
<input type="text" id="name">
<br><br>
<label>Email :</label>
<input type="text" id="email">
<br><br>
<button onclick="submitForm()">Submit</button>
<p id="result"></p>
<script>
function submitForm()
{
   var name=document.getElementById("name").value;
   var email=document.getElementById("email").value;

   if(name!="" && email!="")
   {
      document.getElementById("result").innerHTML=
      "Registration Successful";
   }
   else
   {
      document.getElementById("result").innerHTML=
      "Registration Failed";
   }
}
</script>
</body>
</html>
```

Step 2: Create CSV file with data(StudentData.csv)

```
Ajay,ajay@gmail.com
Rahul,rahul@gmail.com
Anita,anita@gmail.com
```

Step 3 Selenium Code

```
package com.test.selenium;
import java.io.BufferedReader;
import java.io.FileReader;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
public class DataDrivenCSV {
 public static void main(String[] args) throws Exception {
 // Launch Chrome
        WebDriver driver = new ChromeDriver();

        // Maximize browser
        driver.manage().window().maximize();

        // Open HTML page
        driver.get("file:///C:/SeleniumFiles/StudentForm.html");

        // Read CSV file
        BufferedReader br =
                new BufferedReader(
                        new FileReader(
                                "C:\\SeleniumFiles\\StudentData.csv"));

        String line;

        int testCase = 1;

        while ((line = br.readLine()) != null) {

            // Split CSV data
            String[] data = line.split(",");

            // Locate elements
            WebElement name =
                    driver.findElement(By.id("name"));

            WebElement email =
                    driver.findElement(By.id("email"));
```

```
// Clear old values
name.clear();
email.clear();

// Enter data
name.sendKeys(data[0]);
email.sendKeys(data[1]);

// Click Submit
driver.findElement(By.tagName("button")).click();

// Read Result
String result =
      driver.findElement(By.id("result")).getText();

// Display Output
System.out.println("--------------------------------");

System.out.println("Test Case : " + testCase);

System.out.println("Name  : " + data[0]);

System.out.println("Email : " + data[1]);

System.out.println("Expected Result : Registration Successful");

System.out.println("Actual Result   : " + result);

if(result.equals("Registration Successful"))
{
   System.out.println("TEST PASS");
}
else
{
   System.out.println("TEST FAIL");
}

testCase++;

Thread.sleep(1500);
```

```
        }

        br.close();

        driver.quit();

    }

}
```

Output

```
Test Case : 1
Name  : Ajay
Email : ajay@gmail.com
Expected Result : Registration Successful
Actual Result   : Registration Successful
TEST PASS
--------------------------------
Test Case : 2
Name  : Rahul
Email : rahul@gmail.com
Expected Result : Registration Successful
Actual Result   : Registration Successful
TEST PASS
--------------------------------
Test Case : 3
Name  : Anita
Email : anita@gmail.com
Expected Result : Registration Successful
Actual Result   : Registration Successful
TEST PASS
```

Week9

Experiment 7(c): Data Driven Test – Through Front Grids (Web Tables)

Create Html file(web_table)

```
<!DOCTYPE html>
<html>
<head>
   <title>Front Grid Example</title>
</head>
<body>
<h2>Student Data (Front Grid)</h2>
<table border="1" id="studentTable">
<tr>
<th>Name</th>
<th>Email</th>
</tr>

<tr>
<td>Ajay</td>
<td>ajay@gmail.com</td>
</tr>

<tr>
<td>Rahul</td>
<td>rahul@gmail.com</td>
</tr>

<tr>
<td>Anita</td>
<td>anita@gmail.com</td>
</tr>

</table>
<br><br>
<h2>Registration Form</h2>
Name :
<input type="text" id="name">
<br><br>

Email :
<input type="text" id="email">
<br><br>
<button onclick="submitForm()">
Submit
```

```
</button>
<p id="result"></p>
<script>
function submitForm(){
document.getElementById("result").innerHTML=
"Registration Successful";
}
</script>
</body>
</html>
```

Selenium Code

```
package com.test.selenium;

import java.util.List;

import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;

public class DataDrivenFrontGrid {

   public static void main(String[] args)
         throws Exception {

      WebDriver driver = new ChromeDriver();

      driver.manage().window().maximize();

      driver.get("file:///C:/SeleniumFiles/WebTableForm.html");

      // Read all rows except header
      List<WebElement> rows =
            driver.findElements(
            By.xpath("//table[@id='studentTable']/tbody/tr"));

      for(int i=2;i<=rows.size();i++) {

         String name =
               driver.findElement(
               By.xpath("//table[@id='studentTable']/tbody/tr["+i+"]/td[1]"))
               .getText();
```

```
  String email =
                driver.findElement(
                By.xpath("//table[@id='studentTable']/tbody/tr["+i+"]/td[2]"))
                .getText();

        WebElement txtName =
                driver.findElement(By.id("name"));

        WebElement txtEmail =
                driver.findElement(By.id("email"));

        txtName.clear();
        txtEmail.clear();

        txtName.sendKeys(name);
        txtEmail.sendKeys(email);

        driver.findElement(
                By.tagName("button")).click();

        String result =
                driver.findElement(
                By.id("result")).getText();

        System.out.println("--------------------");

        System.out.println("Name : " + name);

        System.out.println("Email : " + email);

        System.out.println(result);

        if(result.equals("Registration Successful"))
           System.out.println("TEST PASS");
        else
           System.out.println("TEST FAIL");

        Thread.sleep(1500);

      }

      driver.quit();

   }

}
```

OutPut

```
Name : Ajay
Email : ajay@gmail.com
Registration Successful
TEST PASS
--------------------
Name : Rahul
Email : rahul@gmail.com
Registration Successful
TEST PASS
--------------------
Name : Anita
Email : anita@gmail.com
Registration Successful
TEST PASS
```

Experiment 7(d): Data Driven Test – Through Excel

Step 1 Create Html file (Studentform.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
</head>
<body>

<h2>Student Registration Form</h2>

<label>Name :</label>
<input type="text" id="name"><br><br>

<label>Email :</label>
<input type="text" id="email"><br><br>

<button onclick="submitForm()">Submit</button>

<p id="result"></p>

<script>
function submitForm(){

var name=document.getElementById("name").value;
var email=document.getElementById("email").value;

if(name!="" && email!="")
{
document.getElementById("result").innerHTML=
"Registration Successful";
}
else
{
document.getElementById("result").innerHTML=
"Registration Failed";
}

}
</script>

</body>
</html>
```

Step 2 Create Excel file (StudentData.xlsx) with Data

Name	Email
Ajay	ajay@gmail.com
Rahul	rahul@gmail.com
Anita	anita@gmail.com
Priya	priya@gmail.com

Step 3: if the Apache POI JAR either by adding Maven Dependency or as external jars

For Maven

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.2.5</version>
</dependency>
```

Step 4 :

```
package com.test.selenium;

import java.io.File;
import java.io.FileInputStream;

 import org.apache.poi.ss.usermodel.*;
 Import org.apache.poi.xssf.usermodel.XSSFWorkbook;

 import org.openqa.selenium.*;
 import org.openqa.selenium.chrome.ChromeDriver;

 public class DataDrivenExcel {

  public static void main(String[] args) throws Exception {

     // Open Excel File
     File file =
           new File("C:\\SeleniumFiles\\StudentData.xlsx");

     FileInputStream fis =
           new FileInputStream(file);

     Workbook workbook =
           new XSSFWorkbook(fis);
```

```
Sheet sheet =
        workbook.getSheetAt(0);

WebDriver driver =
        new ChromeDriver();

driver.manage().window().maximize();

driver.get("file:///C:/SeleniumFiles/StudentForm.html");

// Start from row 1 because row 0 contains headers
for(int i=1; i<=sheet.getLastRowNum(); i++) {

    Row row = sheet.getRow(i);

    String name =
            row.getCell(0).getStringCellValue();

    String email =
            row.getCell(1).getStringCellValue();

    WebElement txtName =
            driver.findElement(By.id("name"));

    WebElement txtEmail =
            driver.findElement(By.id("email"));

    txtName.clear();
    txtEmail.clear();

    txtName.sendKeys(name);
    txtEmail.sendKeys(email);

    driver.findElement(
            By.tagName("button")).click();

    String result =
            driver.findElement(
            By.id("result")).getText();

    System.out.println("------------------------");
    System.out.println("Test Case : " + i);
    System.out.println("Name  : " + name);
    System.out.println("Email : " + email);
    System.out.println("Expected Result : Registration Successful");
    System.out.println("Actual Result   : " + result);

    if(result.equals("Registration Successful"))

       System.out.println("TEST PASS");
    else
       System.out.println("TEST FAIL");
```

```
        Thread.sleep(1000);

      }

      workbook.close();
      fis.close();

      driver.quit();

   }

  }
```

Output :

```
 Test Case : 1
 Name  : Ajay
 Email : ajay@gmail.com
 Expected Result : Registration Successful
 Actual Result   : Registration Successful
 TEST PASS
------------------------
 Test Case : 2
 Name  : Rahul
 Email : rahul@gmail.com
 Expected Result : Registration Successful
 Actual Result   : Registration Successful
 TEST PASS
------------------------
 Test Case : 3
 Name  : Anita
 Email : anita@gmail.com
 Expected Result : Registration Successful
 Actual Result   : Registration Successful
 TEST PASS
------------------------
 Test Case : 4
 Name  : Priya
 Email : priya@gmail.com
 Expected Result : Registration Successful
 Actual Result   : Registration Successful
 TEST PASS
```

Week10

For Experiments 7(e),7(f) ,8 Testng is required to install

For Testng jars update this Dependency in pom.xml

```
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.11.0</version>
</dependency>
```

For Testng launcher if not installed in eclipse
In help-→Go to marketplace-→ search for Testng and install it

Experiment 7(e): Batch Testing – Without Parameter Passing (Suite Run)

Step 1 Create 3 Test Classes

Class 1

```
package com.test.selenium;
import org.testng.annotations.Test;
public class TestBing {
        @Test
        public void gmailTest()
{

            System.out.println("Bing test executed");

        }

    }
```

Class 2

```
package com.test.selenium;
import org.testng.annotations.Test;


 public class TestGoogle {

      @Test
        public void googleTest() {

            System.out.println("Google Test Executed");

        }

    }
```

Class 3

```
package com.test.selenium;
import org.testng.annotations.Test;


public class TestYahoo {


        @Test
        public void youtubeTest() {

            System.out.println("Test yahoo executed");

        }

    }
```

Step 2 Create testing.xml file

Right click on your java project --→ new----→ file----→give file name (testing.xml) --→finish

Paste this in testing.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="BatchSuite">
 <test name="WithoutParameterPassing">
    <classes>
        <class name="com.test.selenium.TestBing"/>
        <class name="com.test.selenium.TestGoogle"/>
        <class name="com.test.selenium.TestYahoo"/>
    </classes>
 </test>
</suite>
```

Now after saving this xml -→ right click on testing.xml in project explorer--→Run as Testng suite

Output

```
 Bing test executed
Google Test Executed
Test yahoo executed

===============================================
BatchSuite
Total tests run: 3, Passes: 3, Failures: 0, Skips: 0
===============================================
```

Experiment 7(f): Batch Testing – With Parameter Passing (Suite Parameters)

Step 1 Create java class TestParameter.java

```
package com.test.selenium;

import org.testng.annotations.Parameters;
import org.testng.annotations.Test;

public class TestParameter {

  @Parameters({"username","password"})
  @Test
  public void login(String user, String pass) {

     System.out.println("Username : " + user);
     System.out.println("Password : " + pass);

     if(user.equals("admin") && pass.equals("admin123")) {

        System.out.println("TEST PASS");

     } else {

        System.out.println("TEST FAIL");

     }

  }
}
```

Step 2 Create new xml file testng1.xml

Right click on your java project --→ new----→ file----→give file name (testing1.xml) --→finish

Paste this in testing1.xml

```
 <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
```

```
<suite name="ParameterSuite">
 <test name="LoginTest">
    <parameter name="username" value="admin"/>
    <parameter name="password" value="admin123"/>
    <classes>
       <class name="com.test.selenium.TestParameter"/>
    </classes>
 </test>
</suite>
```

Now after saving this xml -→ right click on testing1.xml in project explorer--→Run as Testng suite

Output:

```
Username : admin
Password : admin123
TEST PASS

===============================================
Parameter Suite
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
```

Week 11

Experiment 8: Data Driven Batch (Batch + Multiple Data Sets)

Step 1 Create java class LoginTest.java

```
package com.test.selenium;

import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class LoginTest {

  @DataProvider(name="LoginData")

  public Object[][] getData() {

     return new Object[][] {

           {"admin","admin123"},

           {"student","student123"},

           {"guest","guest123"}

     };

  }

  @Test(dataProvider="LoginData")

  public void login(String user,String pass) {

     System.out.println("Username : " + user);

     System.out.println("Password : " + pass);

     System.out.println("Login Successful");

     System.out.println("----------------------");

  }

}
```

Output:

```
Username : admin
Password : admin123
Login Successful
---------------------

Username : student
Password : student123
Login Successful
---------------------
Username : guest
Password : guest123
Login Successful
---------------------
PASSED: com.test.selenium.LoginTest.login("admin", "admin123")
PASSED: com.test.selenium.LoginTest.login("student", "student123")
PASSED: com.test.selenium.LoginTest.login("guest", "guest123")

===============================================
   Default test
   Tests run: 1, Failures: 0, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 3, Passes: 3, Failures: 0, Skips: 0
===============================================
```

Weak 12

Experiment 9: Silent Mode Test Execution (Headless)

```
package com.test.selenium;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class SilentModeExecution {

  public static void main(String[] args) {

    // Create Chrome options
    ChromeOptions options = new ChromeOptions();

    // Enable Headless Mode
    options.addArguments("--headless=new");

    // Optional: Set browser window size
    options.addArguments("--window-size=1920,1080");

    // Launch Chrome in Headless Mode
    WebDriver driver = new ChromeDriver(options);

    // Open Google
    driver.get("https://www.google.com");

    // Find search box
    WebElement searchBox = driver.findElement(By.name("q"));

    // Enter text
    searchBox.sendKeys("Selenium WebDriver");

    // Get page title
    String title = driver.getTitle();

    System.out.println("Page Title : " + title);

    // Checkpoint
    if (title.contains("Google")) {

      System.out.println("TEST PASS");

    } else {
```

```
System.out.println("TEST FAIL");

    }

      // Close browser
      driver.quit();
   }
}
```

OutPut

```
Page Title : Google
TEST PASS
```